

Rapport TD n°2 : Initiation à l'algorithmique

Derrez Lorenzo

September 16, 2026

Exercice 1

Question 1

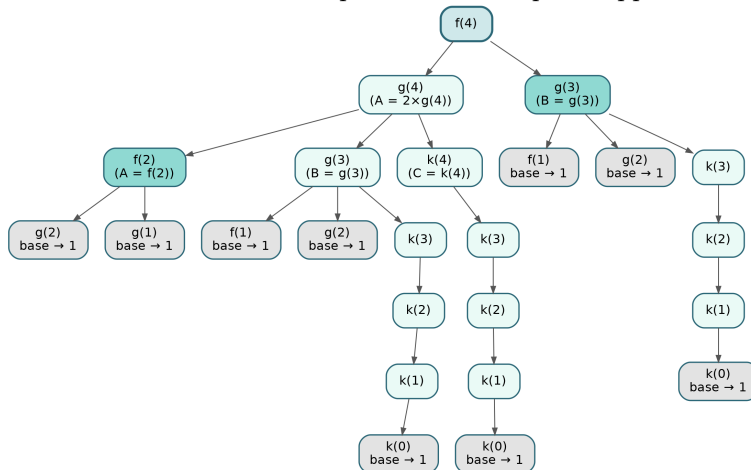
- On a pour le premier tableau :
 $T = a(T)$
 $a(T)$
 $b(T)$
- Les trois propositions sont par ailleurs équivalentes

Question 2

- On a pour le deuxième tableau :
 $a(T); a(T)$
 $b(T); b(T)$
 $a(a(T))$
 $b(a(T))$
- Les quatre propositions sont équivalentes entre elle, on préfèrerait cependant écrire $b(T); b(T)$ par lisibilité

Exercice 2

Voici les différents états dans lequel se trouve la pile d'appel lors de l'exécution de f sur l'entrée 4 :

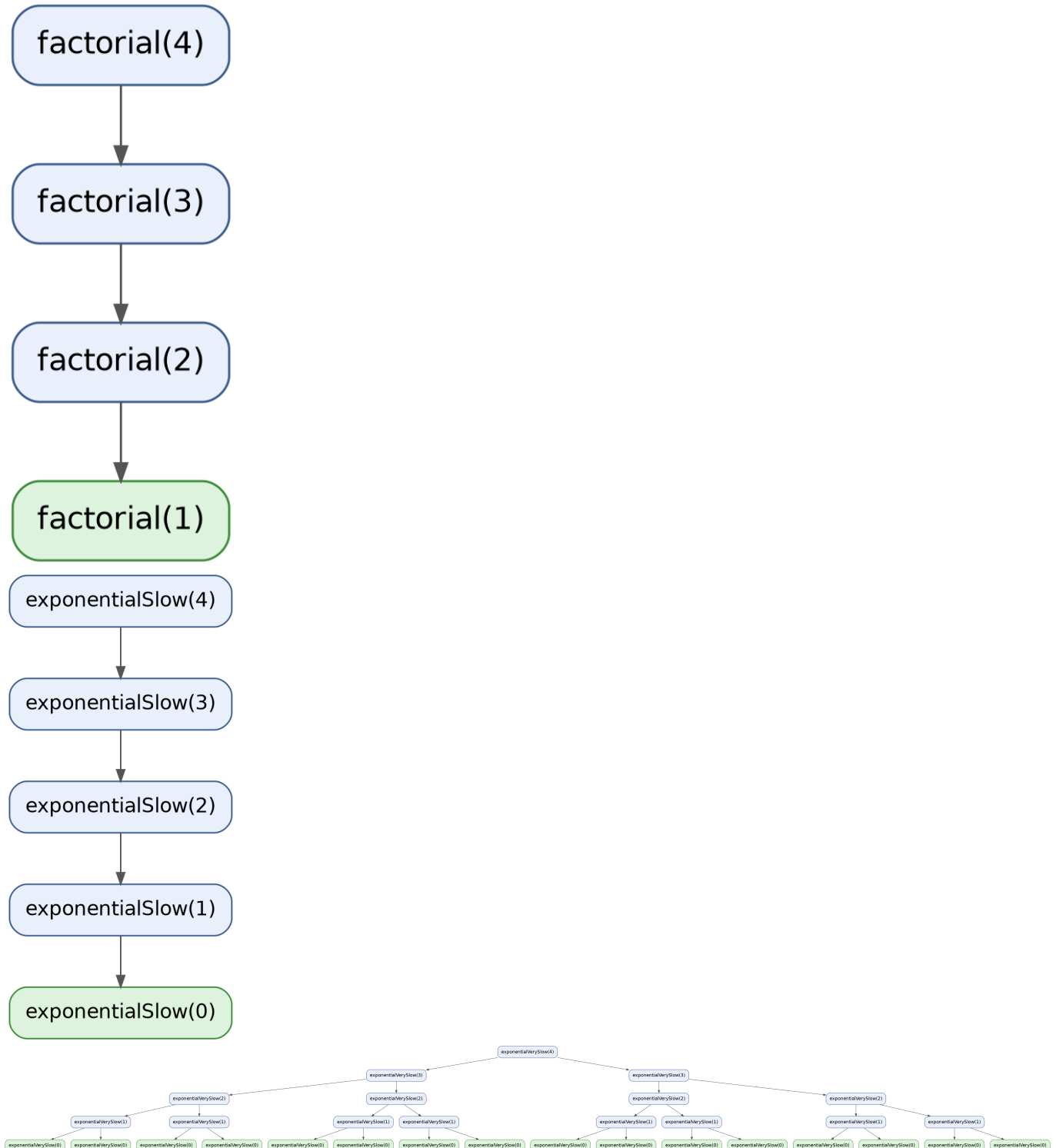


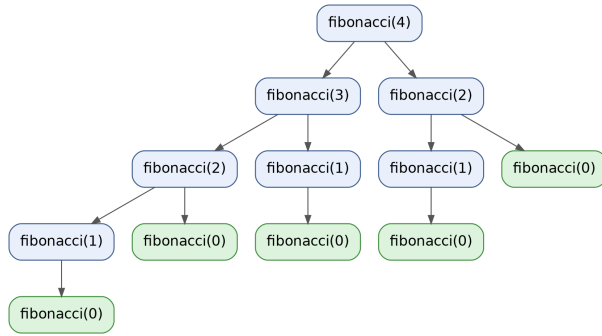
Alors :

- Le nombre total d'états de la pile est : $2 \times$ (nombre total d'appels de fonctions exécutées).
- Résultat : 48 états.

Exercice 3

Voici les différents arbres d'appels que l'on peut dessiner par rapport aux 4 fonctions :





Exercice 4

Factorial(n) :

1. Hauteur : $n-1$
2. Pile max : n
3. Nombre d'appels : n
4. Nombre de noeuds : n

exponentialSlow(n) :

1. Hauteur : n
2. Pile max : $n+1$
3. Nombre d'appels : $n+1$
4. Nombre de noeuds : $n+1$

exponentialVerySlow(n) :

1. Hauteur : n
2. Pile max : $n+1$
3. Nombre d'appels : $2^{n+1}-1$
4. Nombre de noeuds : $2^{n+1}-1$

Factorial(n) :

1. Hauteur : n
2. Pile max : $n+1$
3. Nombre d'appels :
4. Nombre de noeuds :

Exercice 5

Question 1 :

On définit la proposition $\mathcal{P}(n)$ par :

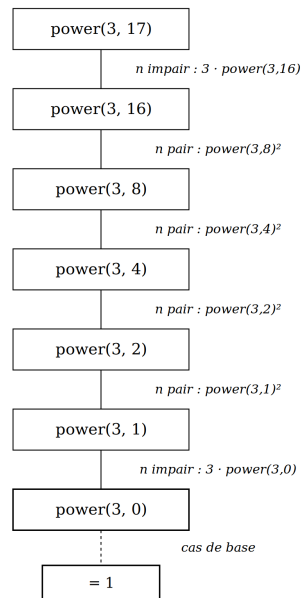
$$x^n = \begin{cases} 1 & \text{si } n = 0 \\ x \times x^{n-1} & \text{si } n \text{ est impair} \\ (x^{n/2})^2 & \text{si } n \text{ est pair, } n > 0 \end{cases}$$

Question 2 :

Algorithm 1 *power*

```
1: Entrée :  $x \in \mathbb{Z}, n \in \mathbb{N}$ 
2: Sortie :  $x^n$  un entier
3: function POWER(x,n)
4:   Si ( $n = 0$ ) alors
5:     return 1
6:   Sinon Si ( $n$  est impair) alors
7:     return  $x * \text{power}(x, n - 1)$ 
8:   Sinon
9:     return  $(\text{power}(x, n/2))^2$ 
10: end function
```

Question 3 :



Question 4 :

Soit $\mathcal{C}(n)$ le nombre total d'appels (avec l'appel initial), on a :

- $\mathcal{C}(0) = 1$
- $\mathcal{C}(n) = \mathcal{C}(n-1) + 1$ si n est impair

- $\mathcal{C}(n) = \mathcal{C}(n/2) + 1$ si n est pair

En comptant sur l'exemple $n = 17$ on obtient : **7 appels**.

Question 5 :

- **ExponentialSlow** effectue $n+1$ appels.
- **ExponentialVerySlow** en effectue $2^{n+1} - 1$

Exercice 6

- Il semblerait que la fonction *Fibonacci* se rapproche de l'écriture de la fonction *ExponentialVerySlow* (Par ailleurs, cela se voit en étudiant les arbres d'appels de l'exercice 3).
- Effectivement, on peut réduire le nombre d'appels de la fonction *Fibonacci* en utilisant le principe de *programmation dynamique*.

On ajoute en paramètre un tableau *memo* qui va stocker les valeurs déjà calculées.

Algorithm 2 Fibonacci par programmation dynamique

```

1: Entrée :  $x$  un entier,  $memo$  un tableau d'entiers
2: Sortie :  $fib(x)$  un entier
3: function FIBO( $x$ ,  $memo$ ) :
4:    $memo[0] \leftarrow 0$ 
5:    $memo[1] \leftarrow 1$ 
6:    $int\ i \leftarrow 2$ 
7:   Tant que ( $i \leq x$ ) faire :
8:      $memo[i] \leftarrow memo[i-1] + memo[i-2]$ 
9:      $i \leftarrow i + 1$ 
10:  return  $memo[x]$ 
11: end function

```
